UneeQ Digital Human Platform
NVIDIA Cloud Native Stack (CNS)

Complete Step-by-Step Deployment on On-Premises GPU Hardware

Powered by NVIDIA NIMs

Version 1.0 | April 2026

Technical Installation Documentation

For On-Premises Installation Specialists

Table of Contents

1. Executive Summary

This document provides comprehensive step-by-step instructions for deploying the UneeQ Digital Human Platform on on-premises NVIDIA GPU hardware using NVIDIA Cloud Native Stack (CNS). CNS MiniPrem enables organizations to run the full digital human stack on hardware they own, without requiring cloud services.

What You Will Deploy

Renny Renderer - UneeQ's digital human rendering engine based on Unreal Engine 5.6
NVIDIA NIMs - Large Language Model (LLM), RAG pipeline, and embedding services
RIVA TTS - NVIDIA's text-to-speech service for voice synthesis
Flowise - Visual workflow builder for AI chatflow orchestration
MicroK8s - Lightweight Kubernetes distribution with GPU support
GPU Operator - Automatic GPU driver management and time-slicing

Cloud vs CNS Comparison

Aspect	Cloud (EKS/AKS/GKE)	CNS (On-Premises)
Infrastructure	Managed by cloud provider	You manage
GPU Availability	Pay per hour	Always available
Data Location	Cloud data centers	On-site
Internet Required	Yes	Only for initial setup
Autoscaling	Yes (dynamic)	No (fixed hardware)
Cost Model	OpEx (ongoing)	CapEx (one-time)

Expected Outcome

Upon completion of this installation guide, you will have a fully functional digital human platform capable of:

- Real-time conversation with AI-powered responses
- Document-based knowledge retrieval (RAG)
- Natural voice synthesis with lip-sync animation
- WebRTC video streaming to client browsers

- GPU time-slicing for multiple concurrent sessions

> **Estimated Installation Time:** 1-2 hours using automated scripts (excluding model download time, which depends on network speed).

2. System Requirements

Operating System

Ubuntu 24.04 LTS is required. The operating system must be a clean image downloaded directly from ubuntu.com. Do not use vendor-customized images as they may have modifications that can cause compatibility issues.

> **IMPORTANT:** Use the standard Ubuntu package manager (apt). Vendor-customized images that replace apt with snap for core packages may cause installation failures. If you encounter issues, start with a fresh Ubuntu 24.04 installation from the official source at ubuntu.com/download/server.

Hardware Requirements

Component	Requirement	Notes
GPU	NVIDIA datacenter GPU	A100, H100, L40, L4, T4, A10G, or DGX systems
VRAM	16GB+ per GPU	More VRAM = more concurrent Renny instances
System RAM	16GB minimum, 32GB+ recommended	Required for LLM inference
Storage	100GB+ SSD (500GB+ NVMe recommended)	For models and persistent volumes
CPU	2+ cores	More cores improve concurrent processing
Network	Internet for initial setup	10Gbps recommended for image pulls

Supported GPUs

Recommended Renny replica counts per GPU (from deploy-local.sh). These assume a full stack deployment with local LLM.

GPU Model	VRAM	Web Mode (stock)	MiniPrem Mode
NVIDIA H100 / A100 80GB	80GB	6	4
NVIDIA A100 40GB	40GB	4	2

GPU Model	VRAM	Web Mode (stock)	MiniPrem Mode
NVIDIA L40 / RTX PRO 6000	48GB	5	3
NVIDIA A10G / L4	24GB	3	2
NVIDIA T4	16GB	2	1

Network Requirements

- Outbound internet access for pulling container images from NVIDIA NGC and Docker Hub
- WebRTC/UDP ports 22000-23000 for video streaming
- TURN/STUN port 3478 for NAT traversal
- HTTPS port 443 for API communications

Required Credentials

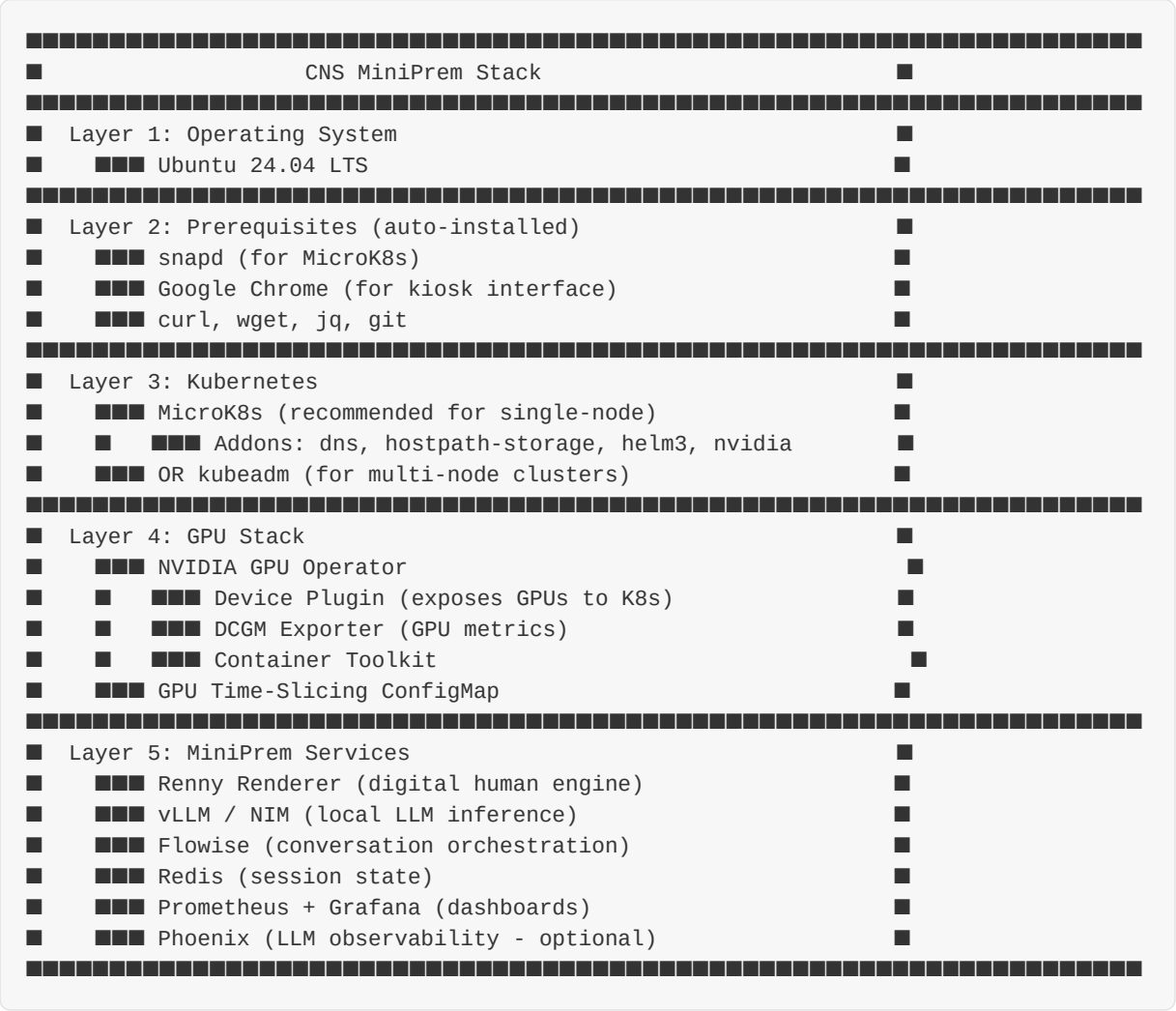
Before beginning installation, obtain the following credentials:

Credential	Value / Source	Purpose
NGC API Key	ngc.nvidia.com/setup/api-key	NVIDIA registry access
Harbor Username	robot$uneeq+dell_isg_validation	UneeQ registry
Harbor Password	(Provided by UneeQ)	UneeQ registry
UneeQ Portal	cdn.enterprise.uneeq.io	Workspace config

3. Architecture Overview

CNS MiniPrem installs the full digital human stack on NVIDIA GPU hardware you own. The platform consists of multiple interconnected services running within a MicroK8s (or kubeadm) Kubernetes cluster.

CNS MiniPrem Stack



Component Overview

Component	Namespace	Purpose
Renny Renderer	uneeq	Digital human rendering, WebRTC streaming, lip-sync
Flowise	uneeq	AI chatflow orchestration, RAG integration
RIVA TTS	riva	Text-to-speech synthesis
LLM NIM	nim-models	Large language model inference (Nemotron)
RAG NIM	nim-rag	Document retrieval and generation pipeline
Embeddings NIM	nim-models	Text embedding for semantic search
GPU Operator	gpu-operator	GPU resource management and time-slicing

Data Flow: Client Browser ↔ DHOP (signaling) ↔ Renny (WebRTC video) → Flowise (conversation) → LLM NIM (AI response) → RIVA (speech synthesis) → Renny (lip-sync)

4. Installation Steps

Phase 1: Prerequisites

This phase covers everything needed before running the deployment scripts.

Step 1.1: Get NGC API Key

An NVIDIA NGC API key is required for downloading AI models:

1. Visit https://ngc.nvidia.com/
2. Sign in or create an account
3. Go to **Setup** → **API Key**
4. Click **Generate API Key**
5. Save the key securely — you will need it during deployment

Step 1.2: Verify GPU Hardware

Ensure your NVIDIA GPU is detected by the system:

```
# Verify GPU is visible
nvidia-smi

# Check GPU device
lspci | grep -i nvidia
```

> **Note:** If nvidia-smi is not found, you need to install NVIDIA drivers first. The deployment scripts can install drivers automatically, or you can install them manually with: sudo add-apt-repository ppa:graphics-drivers/ppa && sudo apt update && sudo ubuntu-drivers autoinstall

Step 1.3: Prepare the Server

Update the system and ensure you have sudo access:

```
# Update system packages
sudo apt update && sudo apt upgrade -y
sudo reboot

# Verify sudo access
sudo whoami  # Should output: root
```

Phase 2: Run Deployment

The deployment scripts automate the entire installation process including Kubernetes setup, GPU operator, time-slicing configuration, and all MiniPrem services.

Step 2.1: Clone the Repository

```
# Clone MiniPrem repository
git clone https://gitlab.com/tgmerritt/miniprem-2025.git
cd miniprem-2025
```

Step 2.2: Choose Deployment Method

CNS MiniPrem supports multiple deployment methods. Choose the one that fits your environment:

Method	Best For	How to Run
Interactive (Recommended)	First-time install, guided setup	./miniprem.sh deploy
Direct (env vars)	CI/CD, automated installs	Set env vars + ./miniprem.sh deploy
Remote (SSH)	Headless servers	kubernetes/scripts/cns/deploy-remote.sh
Ansible	Multi-node, repeatable deploys	ansible-playbook playbooks/cns-install.yml

This guide walks through the **Interactive Deployment** below. For Direct, Remote, and Ansible methods, see Section 5 (Configuration Reference) and Appendix B (File Reference).

Step 2.3: Run the Interactive Deployment

Launch the interactive deployment from the repository root:

```
./miniprem.sh deploy

# The interactive prompts will guide you through:
# Select: 4) NVIDIA Cloud Native Stack (CNS)
# Select: 1) Local Install (or 2 for Remote)
# Select: 1) MicroK8s (recommended for single-node)
# Enter: NGC API Key when prompted
# Enter: DHOP Tenant ID (from UneeQ workspace)
# Enter: DHOP API Key (from UneeQ workspace)
# Select: DHOP Region (US or EU)
# Select: TTS Provider
# Select: Quality level (Web for stock characters, MiniPrem for MiniPrem maps)
# Enter: Number of Renny replicas (or accept GPU-based recommendation)
```

> The script auto-detects your GPU and recommends optimal Renny replica counts based on available VRAM. It also automatically installs prerequisites, sets up Kubernetes, configures GPU time-slicing, deploys the GPU Operator, and installs all MiniPrem services via Helm.

Step 2.4: Kubernetes Distribution Options

Distribution	Best For	Installation
MicroK8s (recommended)	Single-node, easy setup, built-in GPU	snap install via script
kubeadm	Multi-node clusters, flexible networking	kubeadm init via script

Step 2.5: Installation Mode

The script offers two installation modes during interactive setup:

Mode	Includes	Use Case
Minimal	Renny Renderer only	When using external LLM/TTS services
Full Stack	Renny + NIM (local LLM) + Riva TTS	Complete on-premises AI stack

Phase 3: GPU Sizer Tool

The sizer tool calculates how many Renny instances your hardware can support. It runs automatically during interactive deployment, but can also be used standalone.

Step 3.1: Run the Sizer

```
# Auto-detect GPU from current system (from repo root)
./miniprem.sh sizer

# Or call sizer.sh directly with options
kubernetes/scripts/cns/sizer.sh --detect
kubernetes/scripts/cns/sizer.sh --gpu "A100 80GB"
kubernetes/scripts/cns/sizer.sh --gpu "L40"

# List all known GPUs
kubernetes/scripts/cns/sizer.sh --list
```

Step 3.2: Understand the Output

```
╔═══════════════════════════════════════════════════════════════════════════╗
║         MiniPrem CNS Deployment Sizer                                     ║
╚═══════════════════════════════════════════════════════════════════════════╝

GPU Configuration:
  Model: A100 80GB
  VRAM per GPU: 78GB
  GPU Count: 1
  Total VRAM: 78GB

╔═══════════════════════════════════════════════════════════════════════════╗
║ Resolution  ║ Quality  ║ Rennys (no LLM)  ║ Rennys (+ 7B)    ║
╠═══════════════════════════════════════════════════════════════════════════╣
║ 1080p       ║ web      ║ 22 instances     ║ 20 instances     ║
║ 1080p       ║ miniprem ║ 20 instances     ║ 18 instances     ║
║ 4k          ║ web      ║ 14 instances     ║ 13 instances     ║
║ 4k          ║ miniprem ║ 12 instances     ║ 11 instances     ║
╚═══════════════════════════════════════════════════════════════════════════╝
```

Step 3.3: Apply Mode (Optional)

The sizer can also directly apply configuration changes to a running cluster:

```
# Interactive apply - prompts for settings, then applies
./sizer.sh --apply

# Quick apply - auto-detect GPU, use defaults, apply immediately
./sizer.sh --apply-quick
```

Apply mode modifies: GPU time-slicing ConfigMap, GPU Operator cluster policy, Renny deployment replica count, and Renny quality mode.

VRAM Calculation Formula

```
Per Renny VRAM = Base (2.5GB) + (Resolution Overhead x Quality Multiplier)

Resolution Overhead:       Quality Multiplier:
  720p  = 0.5GB              web      = 1.0x
  1080p = 1.0GB              miniprem = 1.3x
  1440p = 1.5GB
  4K    = 3.0GB

Shared Services VRAM:      Max Rennys formula:
  vLLM 7B  = 6GB             (GPU VRAM - Shared Services) / Per Renny VRAM
  vLLM 13B = 10GB
  vLLM 70B = 35GB
  Riva     = 4GB
```

Phase 4: Verify Installation

Step 4.1: Check Cluster Status

```
# Use the built-in status script
./cns/status.sh

# Or manually check:
microk8s kubectl get nodes
microk8s kubectl get pods -A
```

Step 4.2: Verify GPU Configuration

```
# Check GPU allocation on nodes
microk8s kubectl describe node <your-node-name> | grep -A 10 "Allocatable"

# Verify nvidia.com/gpu shows the time-sliced count
# Expected output for 2x L40 with 24 replicas:
#   nvidia.com/gpu: 48

# Run GPU test job
microk8s kubectl apply -f kubernetes/manifests/gpu-test.yaml
microk8s kubectl wait --for=condition=complete job/gpu-test --timeout=300s
microk8s kubectl logs job/gpu-test
microk8s kubectl delete job gpu-test
```

Step 4.3: Verify All Services

```
# Check all namespaces
microk8s kubectl get ns

# Check pods in all relevant namespaces
microk8s kubectl get pods -n gpu-operator
microk8s kubectl get pods -n nim-models
microk8s kubectl get pods -n nim-rag
microk8s kubectl get pods -n riva
microk8s kubectl get pods -n uneeq

# All pods should show STATUS: Running
# All pods should show READY: X/X (all containers ready)
```

Step 4.4: Verify Service Endpoints

```
# List all services
microk8s kubectl get svc -A

# Verify key endpoints:
# - LLM: llama-3-3-nemotron-super-49b-1-5.nim-models.svc.cluster.local:8000
# - RIVA: riva-service.riva.svc.cluster.local:9000
# - Flowise: flowise-service.uneeq.svc.cluster.local:3000
```

5. Configuration Reference

Environment Variables

Variable	Default	Description
NGC_API_KEY	(required)	NVIDIA NGC API key for model downloads
CNS_K8S_TYPE	microk8s	Kubernetes distribution: microk8s or kubeadm
CNS_DEPLOY_TYPE	local	Deployment type: local or remote
CNS_INSTALL_MODE	interactive	Install mode: minimal or full
CNS_QUALITY_LEVEL	miniprem	Quality: web (stock characters) or miniprem (MiniPrem character maps)
RENNY_REPLICAS	auto-calculated	Number of Renny instances to deploy
GPU_TIMESLICE_REPLICAS	8	GPU time-slice replicas per physical GPU
CNS_REMOTE_HOST	-	Remote server IP/hostname (for remote deploy)
CNS_REMOTE_USER	ubuntu	SSH username for remote deploy
CNS_SSH_KEY	~/.ssh/id_rsa	SSH private key path

Helm Values (renny-values-cns.yaml)

```
# Core settings
image: "cr.uneeq.io/uneeq/renny-renderer:enterprise-latest"

deployment:
  totalReplicas: 4        # Number of Renny pods

# GPU time-slicing
gpuTimeSlicing:
  enabled: true
  replicasPerGpu: 4       # Rennys per physical GPU

# Resources per Renny
resources:
  requests:
    nvidia.com/gpu: 1
    memory: "8Gi"
    cpu: "2500m"
  limits:
    nvidia.com/gpu: 1
    memory: "8Gi"
    cpu: "2500m"

# Quality settings
env:
  - name: RENNY_QUALITY_LEVEL
    value: "web"          # web (stock characters) or miniprem (MiniPrem maps)

# Local LLM (NIM)
nim:
  enabled: true
  endpoint: "http://localhost:8000/v1"
  model: "meta/llama-3.1-8b-instruct"

# Telemetry
telemetry:
  enabled: true
  platform: "cns"
```

Quality Mode Details

Each quality mode uses a different type of character map. Choose based on your digital human assets, not as a quality preference.

Mode	RENNY_QUALITY_LEVEL	Use Case	GPU Usage
Web (default)	web	Standard/stock digital humans using UneeQ stock character maps	Lower per Renny (more concurrent sessions)
MiniPrem	miniprem	MiniPrem-specific character maps ONLY	Higher per Renny (fewer concurrent sessions)

> **IMPORTANT:** Only use 'miniprem' quality if you have MiniPrem-specific character maps. Standard/stock UneeQ digital humans must use 'web' quality. Using the wrong mode will result in incorrect rendering.

TTS Configuration Options

Configure text-to-speech in the Helm values or UneeQ persona settings. Pick one provider:

Provider	Key Variables	Notes
NVIDIA Riva (local)	RIVA_URL, RIVA_USE_SSL	gRPC endpoint, e.g. localhost:50051
Azure Speech	AZURE_REGION, AZURE_SPEECH_KEY	Cloud-based, requires API key
ElevenLabs	ELEVEN_LABS_API_KEY, MODEL_ID	Cloud-based, premium voices

GPU Time-Slicing ConfigMap

Applied automatically during installation. To modify after deployment:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: time-slicing-config
  namespace: gpu-operator
data:
  any: |-
    version: v1
    sharing:
      timeSlicing:
        resources:
          - name: nvidia.com/gpu
            replicas: 4    # Adjust based on GPU VRAM
```

6. UneeQ Platform Configuration

With all services deployed, configure the UneeQ platform to connect the digital human to your AI backend through the admin portal.

6.1 Workspace Setup

Log into the UneeQ Admin Portal at **cdn.enterprise.uneeq.io** and navigate to your workspace:

1. Navigate to your workspace
2. Note the **Workspace ID** — this is your Tenant ID
3. In the Security section, click **"+"** next to API Keys to create a new API key
4. Copy and securely store the generated API key
5. Ensure **Enable Private Rendering Pool** is toggled ON
6. Optionally configure Allowed Domains for additional security

6.2 Persona Configuration

Navigate to Personas and create or edit a persona to configure the AI backend connection. Configure the following **API Request Headers**:

Header Name	Value	Description
x-streaming	true	Enable streaming responses
x-chat-id		UUID from your Flowise chatflow
x-chat-endpoint	/api/v1/prediction	Your Flowise API endpoint
Authorization		API key from Flowise

USER-PROVIDED VALUES REQUIRED: The x-chat-endpoint URL depends on your Flowise deployment: - Local Docker: http://localhost:3000/api/v1/prediction - Kubernetes: http://<flowise-service>.<namespace>.svc.cluster.local:3000/api/v1/prediction - Cloud-hosted: Your Flowise cloud URL + /api/v1/prediction Get the Chatflow ID and API Key from your Flowise instance after creating a chatflow.

6.3 Voice/TTS Configuration

Configure the text-to-speech settings to use NVIDIA RIVA TTS. **You must provide your specific TTS URL based on your deployment.**

Setting	Value	Notes
Provider	Build Your Own	Required - enables custom TTS
Voice Name	RIVA	Display name for the voice
TTS URL		User must provide (see below)
TTS API Key		User must provide if required

USER-PROVIDED VALUES REQUIRED: TTS URL: The URL to your RIVA TTS service. Format depends on your deployment: - gRPC: grpc://riva-service.riva.svc.cluster.local:50051 - HTTP: http://<your-server>:8003?uneeq-tpl=... TTS API Key: Authentication key for your TTS service (if required by your deployment).

6.4 Configure renny-values.yaml

Before deploying, edit **kubernetes/values/renny-values.yaml** to set your DHOP credentials. These are obtained from your UneeQ workspace at cdn.enterprise.uneeq.io:

- **renderer.dhop.apiKey** — Your workspace API key (base64 encoded)
- **renderer.dhop.tenantId** — Your workspace Tenant ID (UUID)
- **renderer.tts.proxyUrl** — Should point to your RIVA service

7. Operations & Management (miniprem.sh)

The miniprem.sh wrapper script is the primary CLI for managing your CNS deployment. All day-to-day operations should be run from the repository root directory using ./miniprem.sh.

7.1 Check Status

```
# Comprehensive health report
./miniprem.sh status

# Shows: cluster status, GPU resources, pods, services, storage, summary
```

7.2 Start & Stop Services

```
# Start all CNS services
./miniprem.sh start

# Stop all CNS services
./miniprem.sh stop

# Restart all CNS services
./miniprem.sh restart
```

7.3 Scale Renny Instances

```
# Scale to desired number of Renny instances
./miniprem.sh scale 8

# Quick scale (skips validation, faster)
./miniprem.sh scale-quick 4

# The scale command validates against available GPU slots,
# performs the scale, and waits for rollout
```

7.4 View Logs

```
# View logs for all Renny pods
./miniprem.sh logs

# View GPU sizer recommendations
./miniprem.sh sizer
```

7.5 Upgrade Deployment

```
# Upgrade to latest Helm chart / configuration
./miniprem.sh upgrade

# The upgrade script performs:
#  - Helm upgrade with latest values
#  - Pod restart to pick up changes
#  - Secret clearing if credentials changed
```

7.6 Destroy Deployment

```
# Remove MiniPrem services only (keeps Kubernetes)
./miniprem.sh destroy

# Complete removal including Kubernetes
PURGE_ALL=true ./miniprem.sh destroy
```

> **IMPORTANT:** PURGE_ALL=true will completely remove Kubernetes and all data. Use with caution. Back up any important data before running.

8. Testing & Validation

8.1 Baseline Test (Single Renny)

```
# Deploy 1 Renny
RENNY_REPLICAS=1 ./miniprem.sh deploy

# Wait for pod to be running
microk8s kubectl get pods -n uneeq -w

# Record baseline GPU usage
nvidia-smi --query-gpu=memory.used,utilization.gpu --format=csv -l 1
```

8.2 Validate Sizer Accuracy

```
# Compare sizer prediction vs actual
./miniprem.sh sizer > sizer_prediction.txt

# Run incremental load test
for count in 2 4 6 8 10; do
    echo "Testing with $count Rennys..."
    microk8s kubectl scale deployment/renderer -n uneeq --replicas=$count
    microk8s kubectl wait --for=condition=ready pod -l app=renderer \
      -n uneeq --timeout=300s
    # Check for failures
    microk8s kubectl get pods -n uneeq | grep -E "Error|OOM|CrashLoop"
done
```

8.3 End-to-End Test

Perform a complete end-to-end test of the digital human platform:

1. Open a web browser and navigate to the UneeQ test page for your persona
2. Click to start a session with the digital human
3. Verify the video stream loads (you should see the avatar)
4. Speak a test phrase or type a message
5. Verify: Audio/video stream is working, speech recognition captures input, AI generates relevant responses, text-to-speech produces audio output, and lip sync is accurate to the speech

8.4 Monitoring During Tests

```
# Terminal 1: Watch GPU
watch -n 1 nvidia-smi

# Terminal 2: Watch pods
watch -n 2 'microk8s kubectl get pods -n uneeq'

# Terminal 3: Watch events
microk8s kubectl get events -n uneeq -w

# Terminal 4: Pod logs
microk8s kubectl logs -n uneeq -l app=renderer -f --tail=50
```

9. Troubleshooting

Common Issues and Solutions

Issue: Pods stuck in Pending state

> **Solution:** Check if there are sufficient GPU resources. Run: microk8s kubectl describe pod -n uneeq to see events and resource constraints.

Issue: GPU Not Detected

> **Solution:** Run nvidia-smi to verify GPU is visible. Check lspci | grep -i nvidia. If missing, install NVIDIA drivers with ubuntu-drivers autoinstall.

Issue: Model download fails or times out

> **Solution:** Verify NGC API key is correct. Check network connectivity to nvcr.io. Ensure sufficient storage space for model files.

Issue: Renny fails to connect to DHOP

> **Solution:** Verify kubernetes/values/renny-values.yaml has correct DHOP credentials. Check Renny logs: microk8s kubectl logs -n uneeq -l app=renny

Issue: No audio from digital human

> **Solution:** Verify RIVA service is running. Check TTS URL configuration in persona settings. Test RIVA endpoint directly.

Issue: AI responses are slow or timeout

> **Solution:** Check LLM pod resources. Verify GPU is being utilized: nvidia-smi. Consider adjusting model parameters in kubernetes/values/nim-rag-values.yaml

Issue: Time-Slicing Not Working

> **Solution:** Verify configmap exists: microk8s kubectl get configmap -n gpu-operator. Check cluster policy: microk8s kubectl get clusterpolicy -o yaml

Issue: Out of Memory

> **Solution:** Reduce replicas: microk8s kubectl scale deployment/renderer -n uneeq --replicas=2. Check GPU memory: nvidia-smi pmon -s m

Issue: MicroK8s Not Starting

> **Solution:** Check status: microk8s status. View logs: sudo journalctl -u snap.microk8s.daemon-kubelite -f. Reset: microk8s reset

Log Locations

```
# View logs for specific components
microk8s kubectl logs -n uneeq -l app=renny --tail=100
microk8s kubectl logs -n uneeq -l app=flowise --tail=100
microk8s kubectl logs -n riva -l app=riva --tail=100
microk8s kubectl logs -n nim-models -l app=nim --tail=100

# Follow logs in real-time
microk8s kubectl logs -n uneeq -l app=renny -f
```

Support Contacts

- **UneeQ Support:** support@uneeq.com
- **NVIDIA Enterprise Support:** enterprise-support@nvidia.com
- **UneeQ Documentation:** docs.uneeq.io

Appendix A: GPU Capacity Reference

Expected Renny instances per GPU. These are estimates — actual results vary based on thermal throttling, other workloads, Renny version, and scene complexity.

GPU	1080p Web	1080p MiniPrem	4K Web	4K MiniPrem
T4 (16GB)	2	1	1	1
L4 (24GB)	3	2	1	1
A10G (24GB)	4	3	2	1
RTX PRO 6000 (48GB)	5	3	2	1-2
L40 (48GB)	6	4	3	2
A100 40GB	4	3	2	1
A100 80GB	6	4	3	2
H100 (80GB)	8	6	4	3

Resolution Settings

Resolution	Command Args	VRAM Overhead
720p	-ResX=1280 -ResY=720	0.5GB
1080p	-ResX=1920 -ResY=1080	1.0GB
1440p	-ResX=2560 -ResY=1440	1.5GB
4K	-ResX=3840 -ResY=2160	3.0GB

Appendix B: File Reference

File	Purpose
kubernetes/scripts/cns/deploy-local.sh	Main CNS local installer
kubernetes/scripts/cns/deploy-remote.sh	Remote CNS installer (SSH)
kubernetes/scripts/cns/sizer.sh	GPU capacity calculator
kubernetes/scripts/cns/status.sh	Deployment health checker
kubernetes/scripts/cns/scale.sh	Replica scaler
kubernetes/scripts/cns/cns-update.sh	Configuration updater
kubernetes/scripts/cns/upgrade.sh	Helm upgrade and pod restart
kubernetes/scripts/cns/destroy.sh	Deployment remover
kubernetes/values/renny-values-cns.yaml	Helm values for CNS
kubernetes/ansible/playbooks/cns-install.yml	Ansible installation playbook
kubernetes/ansible/inventory/group_vars/cns.yml	CNS Ansible variables
kubernetes/ansible/vars/cns_versions.yml	Version pinning
kubernetes/manifests/gpu-timeslice.yaml	GPU time-slicing ConfigMap
kubernetes/manifests/gpu-test.yaml	GPU validation Job
kubernetes/manifests/uneeq-stack.yaml	Flowise deployment manifest
kubernetes/values/renny-values.yaml	Renny Helm values
kubernetes/values/riva-values.yaml	RIVA TTS Helm values
kubernetes/values/nim-rag-values.yaml	RAG Blueprint Helm values
kubernetes/charts/riva-api-2.19.0.tgz	RIVA Helm chart
kubernetes/charts/renny-1.0.0.tgz	Renny Helm chart
docker/docker-compose.env	Renny environment config

Appendix C: Command Quick Reference

```
# === MINIPREM.SH WRAPPER (PRIMARY CLI) ===
./miniprem.sh deploy                          # Run interactive deployment
./miniprem.sh status                          # Comprehensive health report
./miniprem.sh start                           # Start all CNS services
./miniprem.sh stop                            # Stop all CNS services
./miniprem.sh restart                         # Restart all CNS services
./miniprem.sh scale <count>                   # Scale Renny instances
./miniprem.sh scale-quick <count>             # Quick scale (skip validation)
./miniprem.sh sizer                           # Calculate GPU capacity
./miniprem.sh logs                            # View Renny logs
./miniprem.sh upgrade                         # Upgrade Helm deployment
./miniprem.sh destroy                         # Remove MiniPrem
PURGE_ALL=true ./miniprem.sh destroy          # Remove everything
```

```
# === STATUS COMMANDS ===
microk8s kubectl get pods -A                  # All pods
microk8s kubectl get svc -A                   # All services
microk8s kubectl get nodes -o wide            # Node info with IPs
nvidia-smi                                    # GPU status
microk8s kubectl describe nodes | grep gpu    # GPU allocation
```

```
# === LOGS ===
microk8s kubectl logs -n <ns> <pod>           # View logs
microk8s kubectl logs -n <ns> <pod> -f        # Follow logs
microk8s kubectl logs -n uneeq -l app=renny --tail=100
```

```
# === DEBUG ===
microk8s kubectl describe pod <pod> -n <ns>  # Pod details
microk8s kubectl exec -it <pod> -n <ns> -- bash  # Shell access
microk8s kubectl get events -n uneeq -w      # Watch events
```

```
# === RESTART COMPONENTS ===
microk8s kubectl rollout restart deployment -n uneeq
microk8s kubectl rollout restart deployment -n riva
microk8s kubectl rollout restart deployment -n nim-models
```

```
# === UNINSTALL (CAUTION) ===
./miniprem.sh destroy                         # Remove MiniPrem only
PURGE_ALL=true ./miniprem.sh destroy          # Complete removal
```